TỔNG QUAN VỀ MÃ REED–SOLOMON

Nguyên lý, thuật toán và ứng dụng trên ESP32-S3 và FPGA

Tài liệu học tập – C nhúng và thiết kế logic số

Tháng 10 năm 2026

Mục lục

1 Reed–Solomon là gì? 1

2 Cấu trúc mã RS(n, k) 1

2.1 Ví dụ RS$(255, 223)$ 2

2.2 Khoảng cách mã và lỗi mất vị trí 2

3 Nền tảng toán học: trường hữu hạn 2

4 Bộ mã hóa (Encoder) 3

5 Bộ giải mã (Decoder) 4

6 So sánh với Checksum và CRC 4

7 Cần phải ôn lại các kiến thức này 5

8 Kết luận 5

1 Reed–Solomon là gì?

Reed–Solomon (RS) là một họ **mã sửa lỗi tiến** (*Forward Error Correction – FEC*). Bộ phát thêm các **symbol kiểm tra** (*parity symbols*) vào dữ liệu gốc; bộ thu sử dụng phần dư thừa đó để phát hiện và, trong giới hạn của mã, **sửa dữ liệu bị sai** mà không cần truyền lại.

Chẳng hạn, một luồng dữ liệu có thể bị nhiễu khi truyền qua UART, đường truyền vô tuyến hoặc khi lưu trữ. CRC chủ yếu cho biết khối dữ liệu *có lỗi hay không*; trong khi đó, Reed–Solomon còn có thể *khôi phục* những symbol sai nếu mức lỗi nằm trong khả năng của mã.

2 Cấu trúc mã RS(n, k)

Một mã RS được mô tả bởi hai tham số:

- k: số symbol dữ liệu gốc;

- n: tổng số symbol của từ mã (*codeword*) sau khi mã hóa.

Số symbol kiểm tra là $r = n - k$. Với mã RS thông thường, số lỗi symbol không biết trước vị trí mà mã có thể sửa được là:

$$\boxed{t = \left\lfloor \frac{n-k}{2} \right\rfloor}. \tag{1}$$

Một *symbol* không nhất thiết là một bit. Với RS trên $\mathrm{GF}(2^8)$, mỗi symbol thường tương ứng một byte (8 bit).

2.1 Ví dụ RS$(255, 223)$

Ta có $n = 255$, $k = 223$, suy ra:

$$\begin{aligned} r &= n - k = 32 \text{ byte}, \\ t &= \left\lfloor \frac{32}{2} \right\rfloor = 16 \text{ symbol}, \\ \eta &= \frac{223}{255} \times 100\% \approx 87{,}45\%. \end{aligned}$$



Tổng: 255 byte

Như vậy, nếu tối đa 16 byte ở các vị trí bất kỳ bị thay đổi trong một từ mã, bộ giải mã lý tưởng có thể sửa được tất cả. Đây là giới hạn theo *số symbol sai*, không phải số bit sai. Nếu số lỗi vượt quá giới hạn, việc sửa đúng không còn được bảo đảm.

2.2 Khoảng cách mã và lỗi mất vị trí

RS là mã *Maximum Distance Separable* (MDS), có khoảng cách Hamming tối thiểu:

$$d_{\min} = n - k + 1. \tag{2}$$

Khi kết hợp e lỗi symbol *không biết vị trí* và v symbol bị mất *đã biết vị trí* (*erasures*), điều kiện sửa lỗi được bảo đảm là:

$$\boxed{2e + v \le n - k}. \tag{3}$$

3 Nền tảng toán học: trường hữu hạn

Nhiều bộ mã RS sử dụng trường hữu hạn $\mathrm{GF}(2^8)$, bao gồm 256 phần tử. Trong trường này:

- Phép cộng và phép trừ đều là phép XOR theo từng bit;
- Phép nhân là phép nhân đa thức trên $\mathrm{GF}(2)$, sau đó lấy dư theo một đa thức bất khả quy bậc 8;

- Với quy ước đa thức $p(x) = x^8 + x^4 + x^3 + x^2 + 1$, biểu diễn nhị phân của đa thức là `0x11D`.

Ví dụ:

$$\texttt{0x53} \oplus \texttt{0xCA} = \texttt{0x99}.$$

Lưu ý rằng phép nhân trên GF(256) **không giống** phép nhân số nguyên thông thường trong ngôn ngữ C.

Listing 1: Ví dụ cộng và nhân trong GF(256) với đa thức 0x11D

```c
#include <stdint.h>

static uint8_t gf_add(uint8_t a, uint8_t b)
{
    return (uint8_t)(a ^ b);
}

static uint8_t gf_mul(uint8_t a, uint8_t b)
{
    uint8_t result = 0;
    for (unsigned i = 0; i < 8; ++i) {
        if (b & 1u) {
            result ^= a;
        }
        uint8_t carry = (uint8_t)(a & 0x80u);
        a = (uint8_t)(a << 1);
        if (carry != 0u) {
            a ^= 0x1Du;  // lower 8 bits of 0x11D
        }
        b >>= 1;
    }
    return result;
}
```

4 Bộ mã hóa (Encoder)

Bộ mã hóa tạo từ mã gồm dữ liệu gốc và parity. Trong cách biểu diễn đa thức *systematic*, đặt $r = n - k$, gọi $m(x)$ là đa thức thông điệp, $g(x)$ là đa thức sinh và

$$p(x) = \left(x^r m(x)\right) \bmod g(x). \tag{4}$$

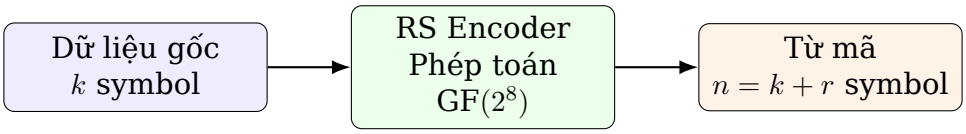
Khi đó từ mã là:

$$\boxed{c(x) = x^r m(x) + p(x)}. \tag{5}$$

Dấu cộng ở đây là phép cộng trong trường đặc số 2, nên tương đương phép trừ. Một dạng đa thức sinh thường gặp là:

$$g(x) = \prod_{i=0}^{r-1} (x - \alpha^{b+i}), \tag{6}$$

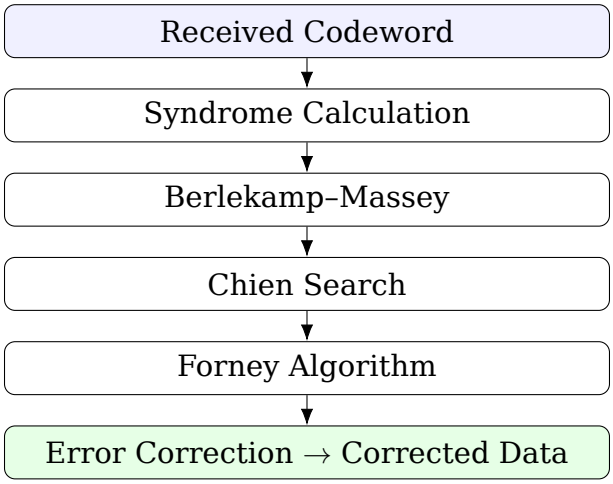
trong đó α là phần tử nguyên thủy của trường, còn b phụ thuộc quy ước của phiên bản RS đang triển khai. Vì vậy, hai hiện thực dùng khác đa thức trường hoặc khác vị trí nghiệm có thể không tương thích.



5 Bộ giải mã (Decoder)

Một quy trình giải mã thường dùng cho RS gồm các khối:

1. **Syndrome Calculation:** đánh giá các syndrome để xác định liệu từ mã có dấu hiệu lỗi hay không.
2. **Berlekamp–Massey:** tìm đa thức định vị lỗi.
3. **Chien Search:** xác định các vị trí symbol bị lỗi.
4. **Forney Algorithm:** tính độ lớn sai lệch của những symbol đó.
5. **Error Correction:** sửa các symbol và xuất dữ liệu đã khôi phục.



Chú ý: Sơ đồ trên là một luồng giải mã điển hình, không phải kiến trúc bắt buộc cho mọi bộ giải mã RS. Các hệ thống thực tế còn có bước kiểm tra kết quả sửa và báo lỗi không sửa được.

6 So sánh với Checksum và CRC

Phương pháp	**Phát hiện lỗi**	**Sửa lỗi**	**Độ phức tạp**
Checksum cơ bản	Có, hạn chế	Không	Thấp
CRC	Có, mạnh với nhiều dạng lỗi	Không	Thấp đến trung bình
Reed–Solomon	Có	Có, trong giới hạn	Cao hơn

Checksum hoặc CRC hữu ích khi hệ thống có thể yêu cầu truyền lại. RS đặc biệt hữu ích khi đường truyền có nhiễu và việc truyền lại tốn kém hoặc không khả thi. RS **không thay thế các cơ chế bảo mật**.

7 Cần phải ôn lại các kiến thức này

1. Ôn phép XOR, dịch bit và biểu diễn đa thức nhị phân.
2. Tự lập trình phép toán cộng, nhân và nghịch đảo trong GF(2^m).
3. Thực hiện Encoder cho RS$(15, 11)$ trên GF(16) để dễ kiểm tra tay.
4. Nghiên cứu syndrome, Berlekamp–Massey, Chien Search và Forney.
5. Viết bộ kiểm thử tạo lỗi ngẫu nhiên và xác nhận giới hạn sửa lỗi.
6. Chuyển kiến trúc sang Verilog/SystemVerilog và kiểm tra bằng testbench.
7. Đo thời gian, bộ nhớ hoặc tài nguyên FPGA và lập báo cáo đánh giá.

8 Kết luận

Reed–Solomon khai thác cấu trúc đại số của các trường hữu hạn để bổ sung thông tin dư thừa và sửa lỗi symbol. Muốn triển khai vững chắc trên ESP32-S3 hoặc FPGA, cần nắm được GF(2^m), Encoder, Decoder và giới hạn $2e + v \leq n - k$, đồng thời đánh giá chi phí tính toán và tài nguyên của hệ thống.